A Survey of Traditional and AI-Augmented Mutation Testing Tools for EShop

Team / Class: Nhóm 12 / Software Testing - CQ2023/3
Members:

MSSV	Thành viên
23120255	Lê Tấn Hiệp
23120262	Tống Dương Thái Hòa
23120264	Nguyễn Phúc Hoàng

Abstract

Mutation testing đánh giá khả năng phát hiện sai khác hành vi của bộ kiểm thử thông qua các thay đổi nhỏ được chủ động đưa vào chương trình. Báo cáo khảo sát các công cụ truyền thống và hướng AI-augmented nhằm lựa chọn bộ công cụ phù hợp với EShop backend JavaScript/Node.js. Các candidate được phân loại theo hệ sinh thái và chức năng, sau đó so sánh theo năm tiêu chí: licence cost, learning curve, EShop fit, AI capability và community. Khảo sát dựa trên tài liệu chính thức, repository của tác giả và các nghiên cứu về mutation testing, assertion synthesis và equivalent mutant detection. StrykerJS được đề xuất làm mutation engine, Claude hỗ trợ đề xuất assertion candidates. Căn cứ lựa chọn là sự phù hợp với SUT, vai trò bổ sung của hai công cụ và khả năng kiểm chứng assertions bằng mutation reports. Đây là kết quả khảo sát tài liệu; hiệu quả thực tế và chất lượng assertions cần được xác nhận trên EShop.

Keywords: mutation testing, test effectiveness, tool survey, StrykerJS, assertion synthesis, large language models, EShop.

1. Introduction

Code coverage cho biết những phần chương trình đã được thực thi bởi tests. Tuy nhiên, việc một nhánh code được chạy không bảo đảm rằng assertions sẽ phát hiện kết quả sai. Mutation testing bổ sung góc nhìn này bằng cách tạo các phiên bản chương trình có thay đổi nhỏ, sau đó kiểm tra xem bộ test có phân biệt được chương trình gốc với phiên bản đã thay đổi hay không [1].

Đối với seminar T10, mục tiêu lựa chọn công cụ là hỗ trợ một quy trình có thể giải thích và tái lập trên EShop backend: chạy baseline, phân tích surviving mutants, bổ sung assertions thủ công, yêu cầu AI đề xuất assertions cho cùng trường hợp và chạy kiểm chứng. Công cụ cần phục vụ cả nghiên cứu lẫn demo, với đầu ra đủ rõ để người học truy nguyên một kết luận đến test và mutant cụ thể.

Báo cáo giải quyết ba câu hỏi:

1. Những nhóm công cụ truyền thống nào có khả năng phục vụ mutation testing trên EShop, và vì sao các lựa chọn khác không được ưu tiên?
2. AI có thể tham gia ở bước nào của quy trình, và các hướng đó khác nhau thế nào về đầu vào, đầu ra và cách kiểm chứng?
3. Bộ công cụ nào phù hợp nhất với mục tiêu khảo sát, và candidate dự phòng có thể đảm nhiệm chức năng nào?

Phạm vi đánh giá là khả năng áp dụng công cụ mutation testing và AI-assisted assertion synthesis vào EShop backend JavaScript/Node.js. Các công cụ thuộc hệ sinh thái khác được xem xét để làm rõ trade-offs và căn cứ lựa chọn.

2. Background and Related Work

2.1. Mutation Testing and Test Oracles

Mutation operator mô tả một loại thay đổi, chẳng hạn thay toán tử so sánh, đảo giá trị Boolean hoặc loại bỏ một biểu thức. Mutant là phiên bản chương trình được tạo bằng một thay đổi cụ thể. Nếu tests hợp lệ trên chương trình gốc nhưng thất bại khi mutant hoạt động, mutant được ghi nhận là `Killed`. Nếu tests vẫn thành công, mutant có thể chỉ ra thiếu dữ liệu kiểm thử, thiếu assertion hoặc một thay đổi không tạo ra khác biệt quan sát được [1], [3].

Ví dụ, thay điều kiện `amount >= threshold` thành `amount > threshold` làm thay đổi hành vi tại đúng ngưỡng. Test chỉ dùng giá trị lớn hơn ngưỡng có thể bỏ sót mutant; test dùng giá trị bằng ngưỡng và assertion về kết quả nghiệp vụ có khả năng phân biệt hai phiên bản. Đây là ví dụ minh họa, không xác lập một quy tắc giảm giá cụ thể của EShop.

Test oracle là căn cứ xác định kết quả mong đợi, được xây dựng từ đặc tả và quy tắc nghiệp vụ. Mutation engine thực thi và phân loại kết quả, nhưng không tự xác nhận expected result của assertion là đúng. Vì vậy, test thất bại trên mutant chỉ có ý nghĩa khi test đã được review về nghiệp vụ và chạy hợp lệ trên code gốc.

2.2. Traditional Mutation Testing

Các công cụ truyền thống sử dụng mutation operators được định nghĩa trước. Khác biệt chính nằm ở hệ sinh thái, mức biểu diễn code được xử lý, cách lựa chọn tests và cơ chế báo cáo. PIT phục vụ JVM; mutmut và Cosmic Ray phục vụ Python; Mull sử dụng LLVM; StrykerJS phục vụ JavaScript/TypeScript [3]-[7].

Chi phí thực thi là một giới hạn quan trọng của mutation testing. Nghiên cứu tại Google cho thấy việc lựa chọn phần code và mutants có ích trong quy trình review là một hướng thực tế để kiểm soát chi phí và tăng tính hữu dụng của kết quả [2]. Đối với EShop, bài học được vận dụng ở mức giới hạn phạm vi nghiệp vụ và phân tích từng survivor. Không suy ra rằng một công cụ sẽ chạy nhanh hơn công cụ khác khi chưa có benchmark cùng điều kiện.

2.3. AI-Augmented Mutation Testing

AI có thể tham gia ở bốn vai trò khác nhau:

Table 1. AI roles in the mutation testing workflow.

AI Role	Input and Output	Verification
Assertion / test synthesis	Nhận spec, code, tests và surviving mutant; đề xuất test data hoặc assertions	Review expected behavior; chạy tests trên code gốc và cùng mutant
Mutant generation	Nhận code và context; đề xuất thay đổi để tạo mutants	Kiểm tra tính hợp lệ và khác biệt hành vi; thực thi từng mutant
Orchestration	Agent hoặc MCP bridge kết nối việc đọc report, sửa tests và chạy lệnh	Đối chiếu diff, commands, logs và reports thực tế
Equivalent mutant analysis	Phân tích khả năng chương trình gốc và mutant tương đương về hành vi	Cần lập luận về phạm vi đầu vào và hành vi quan sát được; AI output là giả thuyết

MuTAP sử dụng surviving mutants để bổ sung thông tin cho prompt sinh tests, tạo cơ sở nghiên cứu cho vòng lặp mutation feedback và AI test generation [17]. LLMorpheus đại diện cho hướng dùng LLM sinh mutants rồi đánh giá bằng StrykerJS đã sửa đổi [14]. ACH của Meta kết hợp sinh mutants theo mối quan tâm cụ thể, sinh tests và phân tích equivalent mutants [18]. Nghiên cứu về equivalent mutant detection cũng đánh giá nhiều chiến lược LLM, trong đó fine-tuned code embeddings là một hướng khác với việc hỏi chatbot qua prompt [19].

Các kết quả nghiên cứu này hỗ trợ việc lựa chọn hướng khảo sát. Chúng không xác nhận hiệu quả của Claude trên EShop và không được dùng làm số liệu thay thế cho thực nghiệm của seminar.

3. Survey Methodology

3.1. Survey Scope and Sources

Khảo sát xem xét hai nhóm: công cụ truyền thống tạo mutants và thực thi tests; công cụ hoặc hướng AI hỗ trợ sinh assertions, sinh mutants, orchestration hoặc phân tích tính tương đương. Các candidate được lựa chọn để đại diện cho những cơ chế và hệ sinh thái liên quan đến mutation testing, ưu tiên khả năng áp dụng vào backend Node.js.

Nhận định về chức năng, ngôn ngữ hỗ trợ và yêu cầu tích hợp được đối chiếu với tài liệu chính thức hoặc repository của tác giả. Bài nghiên cứu gốc được dùng để giải thích cơ chế và giới hạn của các hướng AI. Những nguồn chính được đối chiếu ngày 12 October 2026. Khảo sát đánh giá khả năng phục vụ mục tiêu seminar qua tài liệu; không thực hiện benchmark giữa các công cụ.

3.2. Screening and Comparison

Quy trình phân tích gồm ba bước. Trước hết, các bản ghi được phân loại theo vai trò: mutation engine, test/assertion generator, mutant generator, orchestration hoặc research reference. Tiếp theo, khả năng phục vụ EShop được xét theo ngôn ngữ, đầu ra cần thiết và mức thay đổi môi trường. Cuối cùng, các candidate liên quan được đối chiếu theo năm tiêu chí của tool survey:

Table 2. Comparison criteria.

Criterion	Interpretation for This Survey
Licence cost	Giấy phép phần mềm, khả năng sử dụng miễn phí và chi phí dịch vụ/API bổ sung; không đồng nhất mã nguồn mở với chi phí vận hành bằng 0
Learning curve	Công việc cần học: cấu hình runner, đọc report, chuẩn bị prompt, tích hợp toolchain hoặc kiểm soát agent; đánh giá định tính, chưa đo thời gian học
EShop fit	Khả năng áp dụng vào backend Node.js và bài toán cải thiện assertions; phù hợp ngôn ngữ chưa đủ để xác nhận tích hợp thành công
AI capability	AI thực hiện chức năng nào và đầu ra được kiểm chứng ra sao; công cụ traditional được xét đúng vai trò của nó
Community	Tài liệu, repository, kênh hỗ trợ và mức hỗ trợ được công bố; không dùng mức phổ biến làm bằng chứng về độ chính xác

So sánh được thực hiện định tính theo từng vai trò. Khả năng áp dụng trực tiếp vào SUT và phục vụ đúng bài toán là điều kiện ưu tiên. Một công cụ tốt trong JVM hoặc Python vẫn có thể không phù hợp với EShop. Tương tự, một plugin sinh mutants không mặc nhiên thay thế được công cụ đề xuất assertions.

4. Survey Results

4.1. Traditional Mutation Testing Tools

Table 3. Traditional mutation testing candidates.

Tool	Mechanism and Useful Features	Cost / Setup Considerations	Assessment for EShop
StrykerJS [3]	Mutation testing cho JS/TS; runner plugins; HTML/JSON reports; coverage analysis và incremental mode	Apache-2.0; cần chọn phiên bản tương thích Node và thiết lập tests/runner	Phù hợp trực tiếp về hệ sinh thái. Candidate chính để phân tích survivor và kiểm chứng assertions

Tool	Mechanism and Useful Features	Cost / Setup Considerations	Assessment for EShop
PIT [4]	Mutation testing cho JVM; tích hợp Maven; HTML report và history cho repeated runs	Apache-2.0; cần môi trường JVM và project tương ứng	Có giá trị đối chiếu phương pháp. Không thay StrykerJS trực tiếp trên backend Node.js
mutmut [5]	Python mutation testing; lựa chọn tests, tiếp tục run và giao diện duyệt mutants	BSD-3-Clause; cần Python/tests. README yêu cầu fork support, Windows dùng WSL	Phù hợp SUT Python. Không xử lý trực tiếp backend EShop hiện tại
Cosmic Ray [6]	Python mutation testing; hỗ trợ cấu hình chạy local hoặc distributed	Cần cấu hình session, test command và workers khi phân tán	Hữu ích để nghiên cứu orchestration của mutation runs; không phù hợp trực tiếp Node.js
Mull [7]	Mutation testing cho C/C++ dựa trên LLVM bitcode, sử dụng thông tin Clang AST	Cần toolchain LLVM/Clang và cấu hình build phù hợp	Đại diện cho mutation ở mức compiler. Không phục vụ trực tiếp phạm vi EShop
universalmutator [8]	Source rewriting đa ngôn ngữ bằng regex hoặc Comby; có hỗ trợ JavaScript	Cần tự nối lệnh build/test và phân tích mutants theo project	Khả thi về ngôn ngữ, nhưng cần đánh giá riêng mức tích hợp. Chưa có bằng chứng là phương án thay thế sẵn sàng cho EShop

StrykerJS được ưu tiên vì kết nối trực tiếp hệ sinh thái JS/TS với mutation reports và vòng lặp sửa tests. Kết luận này dựa trên tài liệu và mục tiêu seminar. Thời gian chạy, khả năng dựng harness và mức tương thích với phiên bản EShop cụ thể cần được xác nhận khi thực nghiệm [3].

4.2. AI-Augmented Tools and Directions

Table 4. AI-augmented candidates and directions.

Candidate / Direction	Function and Integration	Cost / Support Evidence	Assessment for This Seminar
Chat LLM + Stryker report [9], [10], [17]	Claude hoặc ChatGPT nhận spec, code, tests và mutant; đề xuất assertion candidates để chạy lại	Dịch vụ của nhà cung cấp; có gói miễn phí giới hạn. MuTAP cung cấp căn cứ nghiên cứu cho mutation feedback	Phù hợp bài toán assertion synthesis. Lựa chọn Claude, giữ ChatGPT là candidate dự phòng

Candidate / Direction	Function and Integration	Cost / Support Evidence	Assessment for This Seminar
Coding agent + Stryker	Agent nối các bước đọc code/report, sửa tests và chạy mutation engine	Chi phí, quyền truy cập và hỗ trợ phụ thuộc sản phẩm và harness được chọn	Là hướng tự động hóa workflow; cần đánh giá riêng khả năng kiểm soát lệnh và truy nguyên kết quả trên EShop
stryker-mcp-reporter [16]	Reporter/MCP bridge cung cấp mutation results cho AI clients hoặc agents	README của tác giả ghi MIT; cần thêm MCP client/server. Độ ổn định chưa được nhóm kiểm thử	Có thể hỗ trợ orchestration; không cần thiết cho vòng lặp assertion có người review
stryker-llm-mutator [15]	Bổ sung mutants qua heuristic hoặc LLM bằng extension cho Stryker	Tác giả mô tả là unofficial monkeypatch; LLM mode phụ thuộc provider. Giấy phép chưa xác minh đầy đủ	Thay đổi tập mutants và tăng yêu cầu compatibility. Không thay thế Claude ở bước đề xuất assertion
Mutahunter [13]	LLM-based mutant generation và chạy test command của project; cần Python 3.11+	AGPL-3.0; có yêu cầu API/model. README minh họa project Java	Hướng nghiên cứu mutant generation. Tích hợp EShop cần PoC; chưa phù hợp làm backup cho assertion synthesis
LLMorpheus [14]	Sinh mutant cho npm JS/TS, chuyển sang StrykerJS đã sửa đổi với precomputed mutants	MIT; cần LLM API và bản Stryker tùy biến. Tác giả xác định là research prototype	Gần EShop về ngôn ngữ, nhưng đổi câu hỏi sang chất lượng mutants và tăng chi phí tích hợp
Pynguin + LLM helpers [12]	Python test generation; tài liệu assertion mô tả SIMPLE và MUTATION_ANALYSIS, giữ assertions phân biệt được mutants	Tài liệu học thuật và repository công khai; việc ghép LLM helpers cần xác minh riêng theo phiên bản	Có giá trị về phương pháp chọn assertions. Không chạy trực tiếp trên Node.js
Diffblue Cover [11]	Reinforcement learning sinh unit tests cho Java/Kotlin; có IDE, CLI và pipeline	Sản phẩm thương mại; điều kiện edition/licence cần kiểm tra nếu triển khai	Đại diện AI test generation. Khác stack EShop và không phải LLM assertion assistant cho Node.js

Candidate / Direction	Function and Integration	Cost / Support Evidence	Assessment for This Seminar
Meta ACH [18]	Hệ thống được mô tả trong paper: targeted mutants, test generation và equivalent analysis trên Android Kotlin	Bài nghiên cứu không cung cấp căn cứ về package cài đặt cho EShop	Research reference; không tính như candidate có thể cài và demo trong phạm vi hiện tại

Khác biệt quan trọng là **sinh assertions trên cùng mutants** và **sinh thêm mutants để đánh giá tests** giải quyết hai bài toán khác nhau. Nếu đổi mutation operators hoặc tập mutants khi so sánh manual và AI assertions, mutation scores không còn phản ánh riêng tác động của assertions. Vì vậy, các công cụ mutant generation được giữ trong khảo sát và related work, không đưa vào bộ triển khai của experiment hiện tại.

Phần Pynguin cũng cần được diễn giải theo phiên bản. Trang assertions đã đọc mô tả SIMPLE, MUTATION_ANALYSIS và NONE, không đủ căn cứ để coi LLM assertion generation là tính năng sẵn dùng ở mọi bản Pynguin [12].

4.3. Shortlist Comparison Against the Five Criteria

Table 5. Main traditional tool, selected AI tool and AI backup candidate.

Criterion	StrykerJS	Claude	ChatGPT
Licence cost	Apache-2.0; không có phí licence cho package. Cần tài nguyên máy và thời gian chạy	Dịch vụ độc quyền; có Free plan, giới hạn sử dụng phụ thuộc gói. API không phải điều kiện bắt buộc của workflow đề xuất [9]	Dịch vụ độc quyền; có Free tier với limits. API không phải điều kiện bắt buộc của candidate workflow [10]
Learning curve	Cần học cấu hình mutation, runner, report và trạng thái mutants; chưa đo thời gian học	Có thể dùng giao diện hội thoại; cần chuẩn hóa input và review assertions	Cùng mức yêu cầu phương pháp: cung cấp context, review và thực thi candidates
EShop fit	Hỗ trợ JS/TS; cần xác nhận Node, runner, fixtures và data reset của SUT [3]	Có thể được cung cấp code/spec/mutant EShop để đề xuất assertions; chưa kiểm chứng chất lượng trên SUT	Candidate cho cùng dạng input và output; chưa kiểm chứng chất lượng trên SUT
AI capability	Vai trò trong nghiên cứu là mutation engine và thực thi kiểm chứng	Assertion synthesis và giải thích dự kiến; mức chính xác chỉ xác định sau review/run	Vai trò tương ứng nếu cần thay phần AI; không thay mutation engine

Criterion	StrykerJS	Claude	ChatGPT
Community	Tài liệu, repository và kênh hỗ trợ của dự án [3]	Tài liệu nền tảng và hỗ trợ nhà cung cấp [9]	Help Center và tài liệu của nhà cung cấp [10]

Claude được đề xuất cho vai trò assertion synthesis vì có thể sử dụng trong vòng lặp hội thoại với context giới hạn theo từng mutant, đồng thời lưu rõ đề xuất và phần chỉnh sửa. ChatGPT là candidate cho cùng vai trò. Khảo sát không có thử nghiệm đối đầu hai công cụ với cùng prompt, model, budget và SUT; do đó không kết luận Claude chính xác hơn, rẻ hơn hoặc dễ học hơn ChatGPT. Ưu tiên chọn công cụ có số thành viên đăng ký nhiều nhất.

5. Discussion and Selection Rationale

5.1. Recommended Tool Pair

Bộ triển khai được đề xuất là **StrykerJS + Claude**, với ba căn cứ chính:

1. **Phù hợp SUT và mục tiêu T10.** StrykerJS xử lý JS/TS và cung cấp mutation results cần thiết để nghiên cứu khả năng phát hiện lỗi của tests trên backend Node.js [3].
2. **Vai trò bổ sung và kết quả có thể truy nguyên.** Claude đề xuất assertions; spec và human review xác định expected behavior; StrykerJS cùng test runner thực thi kiểm chứng. Mỗi nhận định có thể gắn với candidate, bản đã sửa và mutant được đánh giá [3], [9], [17].
3. **Giữ điều kiện so sánh rõ ràng.** Có thể dùng cùng production code, mutation configuration và mutants để so sánh baseline, manual assertions và Claude assertions. Hướng này phục vụ trực tiếp câu hỏi test effectiveness, không cần thêm LLM mutant generator hay MCP layer.

Lựa chọn này xác định công cụ và cách đánh giá. Khả năng triển khai thành công, mutation score đạt được và mức cải thiện assertions vẫn cần evidence từ thực nghiệm.

5.2. Tool Limitations and Interpretation Risks

Theo định nghĩa Stryker, `Detected = Killed + Timeout`, `Undetected = Survived + No coverage` và `Valid = Detected + Undetected`. Mutation score của tool là `Detected / Valid × 100`, không đơn giản là `Killed / toàn bộ mutants được sinh` [3]. Khi mẫu số bằng 0, cần ghi không có cơ sở tính score. `Compile error`, `Runtime error`, `Ignored` và `Pending` được báo riêng theo report.

Với câu hỏi assertion effectiveness, bảng kết quả phải bổ sung trạng thái từng mutant và test gây thất bại. Các rủi ro diễn giải cần được kiểm soát như sau:

Table 6. Tool limitations and interpretation risks.

Failure Mode	Consequence	Required Check
Score tăng do Timeout	Có thể bị hiểu nhầm là assertion phát hiện lỗi tốt hơn	Báo `Killed` và `Timeout` riêng; xem test/log của mutant được chọn
Assertion học theo bug hiện tại	Test chạy thành công nhưng chấp nhận hành vi trái spec	Xác định expected result từ nghiệp vụ trước khi chấp nhận candidate
Assertion quá yếu hoặc gọi API không tồn tại	Test không chạy được, hoặc chạy nhưng không phân biệt mutant	Review cú pháp/API, chạy code gốc và đúng mutant mục tiêu
Kết quả incremental bị cũ	Report tái sử dụng kết quả dù dependency, environment hoặc file ngoài phạm vi tracking đã đổi	Chạy lại các mutants trong phạm vi so sánh khi xác nhận kết quả; ghi rõ chế độ run [3]
Survivor bị gán equivalent quá sớm	Khoảng trống tests có thể bị loại khỏi phân tích	Giữ nhãn equivalent candidate cho đến khi có lập luận về hành vi và phạm vi đầu vào
Flaky tests hoặc data không ổn định	Cùng điều kiện dự kiến có thể cho trạng thái khác nhau	Kiểm soát clock, dữ liệu, reset và môi trường; điều tra run bất thường

Mutation operators được định nghĩa theo luật không bảo đảm mọi run là tất định. Tests, timeout, dữ liệu và môi trường vẫn có thể làm kết quả biến động. Tương tự, AI giải thích rằng assertion sẽ kill mutant không phải evidence thực thi. Đánh giá cuối cùng cần kết hợp spec, test results và mutation reports.

6. Threats to Validity

Construct validity. Năm tiêu chí được đánh giá định tính, không phải phép đo accuracy, runtime hoặc khả năng phát hiện lỗi. Phân loại theo vai trò và sự phù hợp với SUT giúp tránh so sánh trực tiếp các công cụ khác chức năng. Learning curve và chất lượng hỗ trợ vẫn cần trải nghiệm thực hành.

Internal validity. Tài liệu công cụ mô tả khả năng sản phẩm, không chứng minh cấu hình EShop đã hoạt động. Nếu production code, test data, mutation set hoặc môi trường thay đổi giữa các runs, kết quả có thể bị ảnh hưởng ngoài assertions. Baseline, run metadata và trạng thái từng mutant cần được giữ rõ khi thực nghiệm.

External validity. Những bài nghiên cứu được dẫn sử dụng ngôn ngữ, datasets, mô hình và workflow khác EShop. Hiệu quả trên Python, Java hoặc Android Kotlin không tự chuyển thành hiệu quả trên backend Node.js. Khảo sát cũng không chứng minh rằng kết quả của một model áp dụng cho mọi phiên bản Claude hoặc ChatGPT.

Source and temporal validity. Đây là khảo sát có phạm vi phục vụ lựa chọn công cụ cho EShop, chưa phải systematic literature review bao quát toàn bộ công cụ hiện có. Version, licence, prices, quotas và trạng

thái bảo trì có thể thay đổi. Những nội dung thiếu nguồn hoặc chưa thử tích hợp được giữ ở mức chưa xác minh.

7. Conclusion

Khảo sát xác định hai phân biệt cần thiết khi lựa chọn công cụ: sự phù hợp với hệ sinh thái SUT và chức năng AI thực hiện trong mutation workflow. PIT, mutmut, Cosmic Ray và Mull đại diện cho các hệ sinh thái khác; các hướng LLM mutant generation, orchestration và equivalent analysis mở rộng nội dung nghiên cứu nhưng không cùng chức năng với assertion synthesis.

Đối với phạm vi EShop hiện tại, **StrykerJS + Claude** là bộ công cụ phù hợp để triển khai nghiên cứu manual và AI-assisted assertions. Căn cứ lựa chọn là khả năng phục vụ đúng mục tiêu, tách rõ trách nhiệm và truy nguyên kết quả. Mức cải thiện thực tế chỉ được kết luận sau các runs có evidence trên cùng baseline.

AI Assistance Disclosure

OpenAI Codex hỗ trợ tổng hợp thông tin, tổ chức cấu trúc và soạn bản thảo. Vai trò, ngôn ngữ hỗ trợ, yêu cầu tích hợp và các giới hạn chính được đối chiếu với tài liệu công cụ và bài nghiên cứu trong References.

References

Các nguồn công cụ và nghiên cứu dưới đây được đối chiếu ngày 12 October 2026. Với repository/README, claim mô tả khả năng do tác giả công bố, không hàm ý nhóm đã kiểm thử sản phẩm.

[1] Y. Jia and M. Harman, “An Analysis and Survey of the Development of Mutation Testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649-678, 2011, doi: 10.1109/TSE.2010.62. Author manuscript.

[2] G. Petrović and M. Ivanković, “State of Mutation Testing at Google,” *ICSE Software Engineering in Practice*, 2018. Google Research.

[3] Stryker Mutator, StrykerJS and Mutation Testing Elements documentation. Introduction, Getting Started, Configuration, Mutant States and Metrics, Incremental.

[4] PIT, official project and documentation. Repository, Maven Quick Start.

[5] mutmut, official repository and README. Mutation Testing System for Python.

[6] Cosmic Ray, official documentation. Overview, Distributed Concurrent Mutation Testing.

[7] Mull, official documentation. Overview, How Mull Works.

[8] A. Groce and contributors, universalmutator repository and README. UniversalMutator.

[9] Anthropic, official product and evaluation documentation. Claude Plans, Define Success Criteria and Build Evaluations.

[10] OpenAI, “ChatGPT Free Tier FAQ.” OpenAI Help Center.

[11] Diffblue, “What is Diffblue Cover?” Official Documentation.

[12] Pynguin, “Generating Assertions,” development documentation. Assertion Generation and Mutation Analysis.

[13] CodeIntegrity AI, Mutahunter repository and README. LLM-Based Mutation Testing.

[14] LLMorpheus authors, research repository and README. LLMorpheus.

[15] hughescr, stryker-llm-mutator repository and README. LLM-Driven Mutation Generator Plugin.

[16] M. Kluth, stryker-mcp-reporter repository and README. Stryker MCP Reporter.

[17] A. Moradi Dakhel et al., “Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing,” arXiv:2308.16557, 2023. MuTAP Paper.

[18] C. Foster et al., “Mutation-Guided LLM-based Test Generation at Meta,” arXiv:2501.12862, 2025. ACH Paper.

[19] Z. Tian et al., “Large Language Models for Equivalent Mutant Detection: How Far Are We?,” *ISSTA*, 2024. Author Preprint.